



REST APIs στην Πράξη

(με JavaScript/PHP)

JWT – OAuth 2.0



API – Application Programming Interface

- ❖ Ένα **API** ορίζει ένα σύνολο κανόνων με τους οποίους διαφορετικά προγράμματα μπορούν να επικοινωνούν μεταξύ τους.

- ❑ Ορίζεται ουσιαστικά μία διεπαφή μεταξύ τους.

❖ Web API

- ❑ Η επικοινωνία γίνεται με αποστολή κατάλληλα διαμορφωμένων URLs (αιτήσεων HTTP(S)).

- Αρχιτεκτονική REST

- ❑ Τα δεδομένα επιστρέφονται σε JSON ή XML.

```
<weather>
  <date>2016-08-27</date>
  <location>Chicago</location>
  <info>Hot</info>"//
</weather>
```

```
{
  "date": "2016-08-27",
  "location": "Chicago",
  "info": "Hot"
}
```

- ❖ Η ύπαρξη ποικιλίας συσκευών αλλά και εφαρμογών με δυνητική δυνατότητα συνεργασίας, έχει κάνει τη χρήση των web APIs ιδιαίτερα δημοφιλή.

- ❑ Διαχωρισμός διεπαφής χρήστη και δεδομένων.

- ❑ Δημιουργία νέων εφαρμογών με αξιοποίηση μιας ή περισσότερων πηγών δεδομένων (άλλων εφαρμογών, π.χ. τύπου mashups).

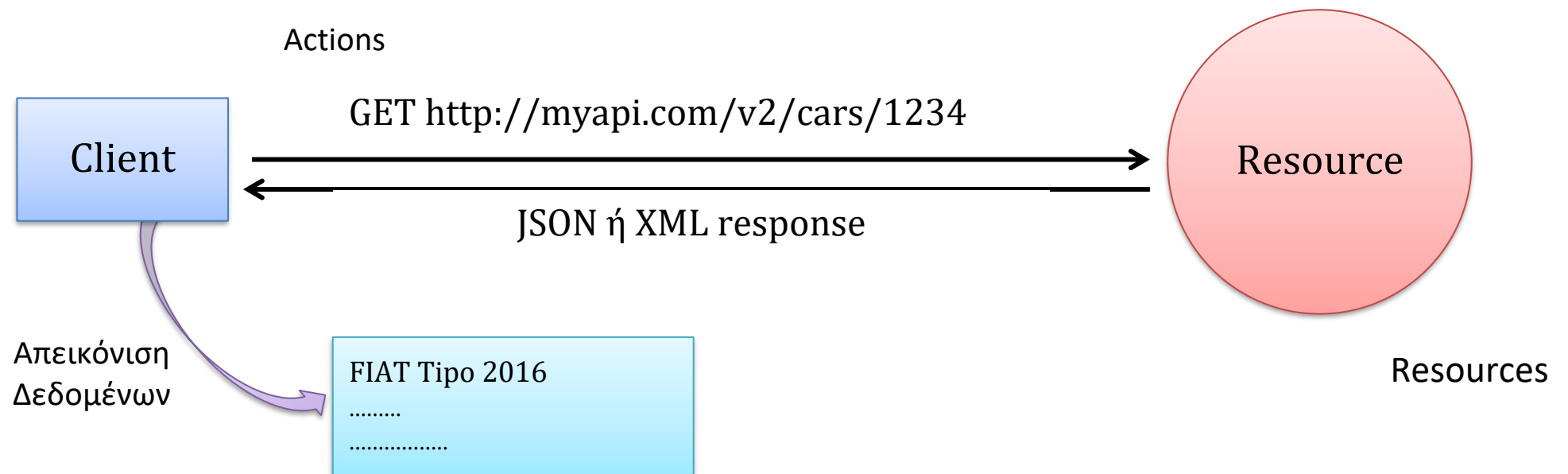
- Βολική προσέγγιση στα ανοιχτά δεδομένα (open data).

- ❑ Ερευνητικοί λόγοι/σκοποί.

Αρχιτεκτονική REST

(REpresentation State Transfer)

- ❖ Αρχιτεκτονική συστήματος βασισμένη στο κλασικό μοντέλο client/server που χρησιμοποιείται για την υλοποίηση εφαρμογών δικτύου.
- ❖ Δεν είναι standard η ίδια - χρησιμοποιεί άλλα standards
 - ❑ HTTP, URL, HTML, XML, JSON
- ❖ Σχεδόν πάντα χρησιμοποιείται το HTTP πρωτόκολλο κυρίως λόγω της ευκολίας στην κατανόηση και της ευχρηστίας του έναντι σε άλλες λύσεις όπως για παράδειγμα το SOAP.



Βασικές Αρχές REST 1/3

❖ **HTTP πρωτόκολλο** – Η χρήση των HTTP verbs είναι μάλλον τυποποιημένη

Resource	POST create	GET read	PUT update	DELETE delete
/dogs	create a new dog	list dogs	bulk update dogs	delete all dogs
/dogs/1234	error	show Bo	if exists update Bo if not error	delete Bo

POST: Δημιουργία πόρου σε server.

GET: Ανάκτηση πόρου από server.

PUT: Ενημέρωση στοιχείων πόρου στον server.

DELETE: Διαγραφή πόρου από τον server.

CRUD λειτουργίες

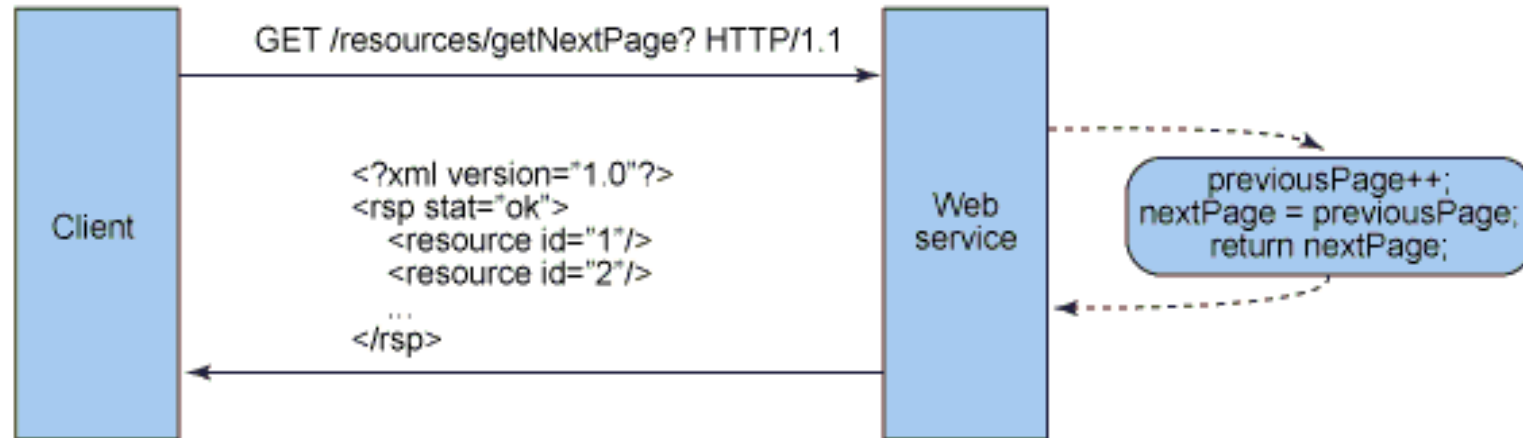
❖ Παραεκκλίσεις από τα παραπάνω είναι κάτι σύνηθες! ==>

HTTP Verb	Path	Action	Used For
GET	/books	index	display all books
GET	/books/new	new	return an HTML form for creating a new book
POST	/books	create	create a new book
GET	/books/:id	show	display a specific book
GET	/books/:id/edit	edit	return an HTML form for editing a book
PUT	/books/:id	update	update a specific book
DELETE	/books/:id	destroy	delete a specific book

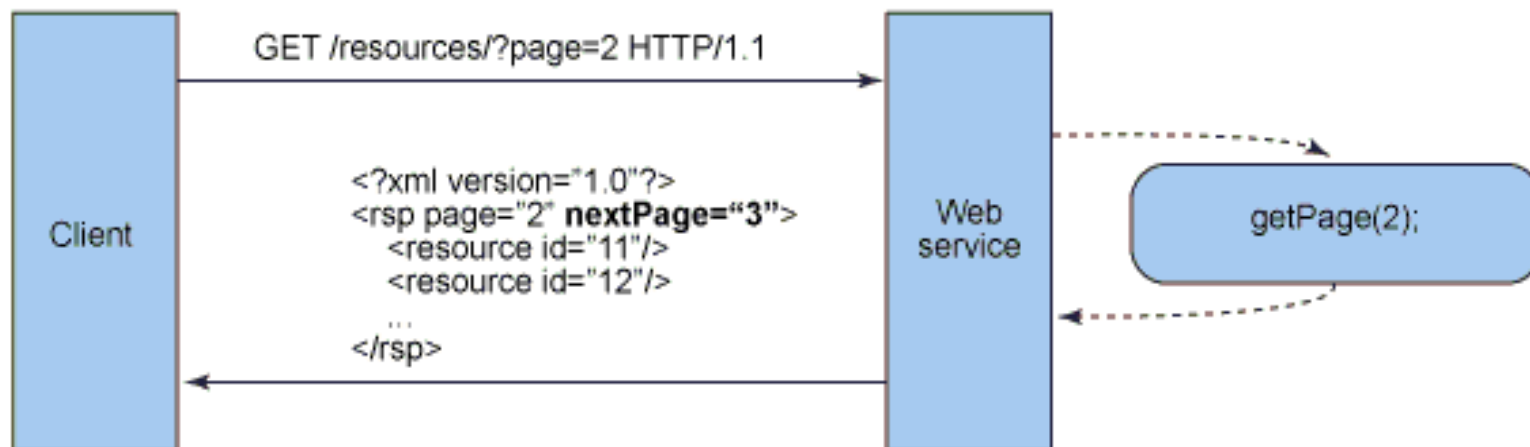
Βασικές Αρχές REST 2/3

❖ Stateless Server-Side

- ❑ **statefull** – ο server θυμάται την κατάσταση επικοινωνίας



- ❑ **stateless** – ο server ΔΕΝ θυμάται την κατάσταση επικοινωνίας (**τότε ποιος;**)



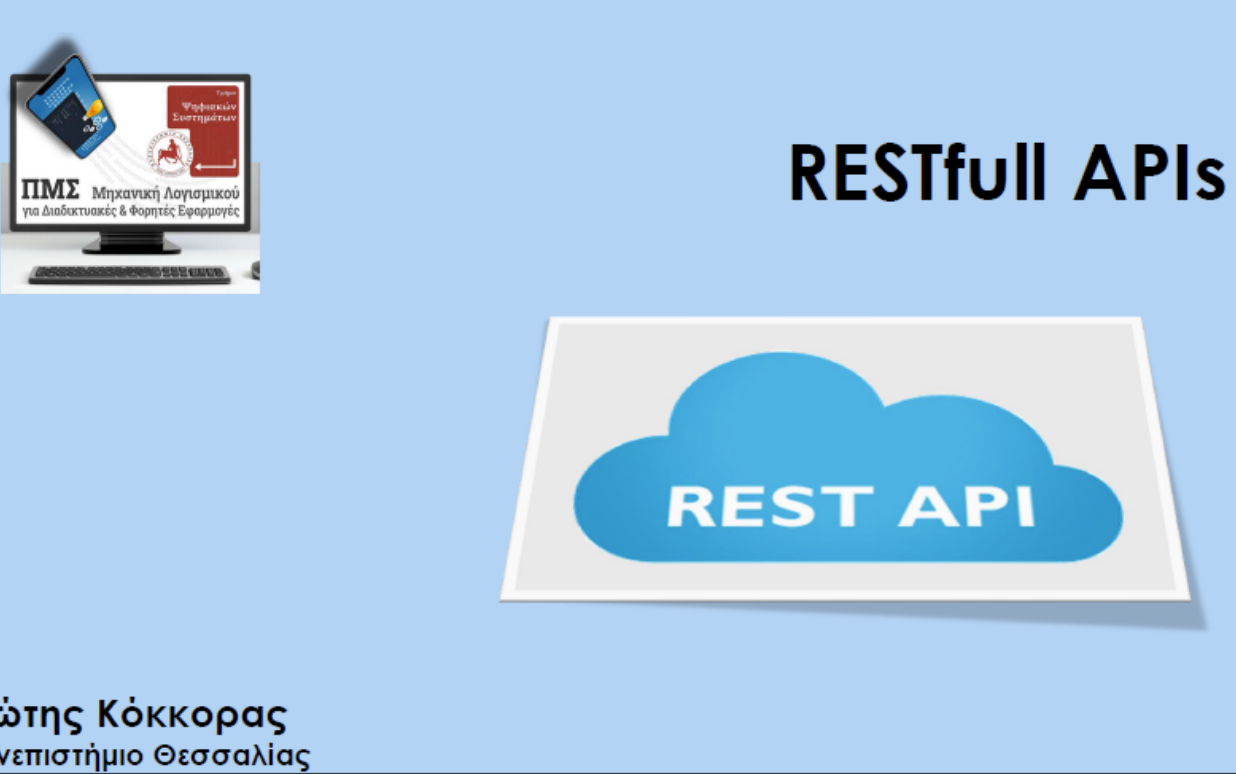
προφανώς ο **client**

Βασικές Αρχές REST 3/3

❖ Δομή στα URIs.

Βλ. καλές πρακτικές στα slides

Προηγμένες Web Εφαρμογές



RESTfull APIs

REST API

Φώτης Κόκκορας
Πανεπιστήμιο Θεσσαλίας

❖ Η μεταφορά δεδομένων γίνεται με JSON (κυρίως) ή XML.

Παράδειγμα GET κλήσης

- ❖ <http://myapi.gr/photos/search/?method=searchByTerms&term=athens&thumbs=yes>
 - ❑ η κλήση γίνεται στο σχετικό php script
 - συνίσταται η χρήση ενός script που μέσα του υλοποιεί όλη την λειτουργικότητα
 - ✓ **api.php**, **index.php**, κτλ
 - συνήθως υποστηρίζονται πολλές ετερογενείς κλήσεις σε επίπεδο URI που όμως κατευθύνονται στο ίδιο php script (π.χ. μέσω .htaccess) – είναι τα λεγόμενα **routes**
 - ✓ τα routes μπορεί να υλοποιηθούν ως απλές callable συναρτήσεις...

```
$app->get('/hello/{name}', function (Request $request, Response $response) {  
    $name = $request->getAttribute('name');  
    $response->getBody()->write("Hello, $name");  
    return $response;  
});
```
 - ✓ ή ως controllers μιας MVC αρχιτεκτονικής
 - ❑ ανάλογα με την κλήση και τυχόν παραμέτρους, γίνεται σύνδεση στη database
 - ❑ επιστροφή αποτελεσμάτων σε JSON ή XML
- ❖ Θα δούμε κάποια παραδείγματα στο SLIM σε άλλο μάθημα.

JSON και JavaScript 1/3

- ❖ Ως υποσύνολό της, τα JSON δεδομένα είναι άμεσα προσβάσιμα με JavaScript

```
myObject = {  
  "first": "John",  
  "last": "Doe",  
  "age": 39,  
  "interests": ["Reading", "Mountain Biking", "Hacking"],  
  "skills": [ { "category": "Javascript", "tests": [ { "name": "One", "score": 90 },  
                                                    { "name": "Two", "score": 96 } ] },  
              ]  
}
```

- ❖ Παραδείγματα προσπέλασης στοιχείων με JavaScript:

- ❑ `myObject.first` θα μας δώσει John
- ❑ `myObject.interests[1]` θα μας δώσει Mounting Biking
- ❑ `myObject.skills[0].category` θα μας δώσει την τιμή Javascript
- ❑ `myObject["skills"][0]["category"]` θα μας δώσει πάλι την τιμή Javascript
- ❑ `myObject["skills"][0]["tests"][1]["score"]` θα μας δώσει 96
- ❑ `myObject.skills[0].tests[1].score` θα μας δώσει 96

JSON και JavaScript 2/3

Πώς μετατρέπουμε τα JSON data σε JavaScript data;

❖ Μετατροπή συμβολοσειράς σε JavaScript αντικείμενο:

❑ με τη συνάρτηση **eval (...)**

```
var myObject = eval( '(' + myJSONtext + ')' );
```

- Χρήση του compiler της Javascript.
- Χρήση παρενθέσεων για αποφυγή ασάφειας στη σύνταξη της γλώσσας.
- Πρόβλημα ασφάλειας: το myJSONtext είναι όντως JSON data? Μήπως είναι κάποιο κακόβουλο JavaScript script?!!

❑ με χρήση ενός JSON parser:

```
var myObject = JSON.parse(myJSONtext) ;
```

- Τα δεδομένα που λαμβάνονται από το server είναι πάντα text. Περνώντας τα από την JSON.parse() γίνονται JavaScript objects.
- Ο parser αναγνωρίζει μόνο JSON text και αγνοεί τα υπόλοιπα.

JSON και JavaScript 3/3

❖ Λήψη JSON data με AJAX κλήση

- ❑ Αν τα δεδομένα είναι array, η parse επιστρέφει JavaScript array και όχι object.

```
var xmlhttp = new XMLHttpRequest();
xmlhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        myArr = JSON.parse(this.responseText);
        document.getElementById("demo").innerHTML = myArr[0];
    }
};
xmlhttp.open("GET", "json_demo_array.txt", true);
xmlhttp.send();
```

- όπου το json_demo_array.txt περιέχει: ["Ford", "BMW", "Audi", "Fiat"]

❖ Δεδομένα που δεν συμφωνούν με τους αναγνωρίσιμους τύπους της JSON γλώσσας, πρέπει να μετατραπούν κατάλληλα!

- ❑ π.χ. παραγωγή date object από string ημερομην

```
var text = '{ "name":"John", "birth":"1986-12-14", "city":"New York"}';
var obj = JSON.parse(text) {
    obj.birth = new Date(obj.birth);

    document.getElementById("demo").innerHTML = obj.name + ", " + obj.birth;
```



JSON και jQuery AJAX

- ❖ Αν προτιμάτε την jQuery έναντι της απλής JavaScript:

```
$.ajax(  
    { url: url,                //sent data to server via url parameters  
      dataType: 'json',  
      data: data,              //get data back  
      success: callback       //call function when the call is done  
    }  
);
```

JSON και PHP

❖ **Αποκωδικοποίηση** JSON object: `json_decode($json_string, $assoc)`

❑ `$assoc`: με true μας επιστρέφει τα αποτελέσματα σε associative πίνακα

❑ Παράδειγμα (<http://php.net/manual/en/function.json-decode.php>):

```
$json = '{"a":1, "b":2, "c":3, "d":4, "e":5}';
```

```
var_dump(json_decode($json, true));
```

- μας δίνει ως αποτέλεσμα

```
array(5) { ["a"] => 1 ,  
           ["b"] => 2,  
           ["c"] => 3,  
           ["d"] => 4,  
           ["e"] => 5  
}
```

❖ **Κωδικοποίηση** JSON object (από PHP array): `json_encode(value)`

❑ Παράδειγμα:

```
$arr = array('a' => 1, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 5);
```

```
echo json_encode($arr);
```

- μας δίνει ως αποτέλεσμα:

```
{"a":1,"b":2,"c":3,"d":4,"e":5}
```

JSON και αποφυγή της PHP !!!

- ❖ Τα σύγχρονα RDBMS έχουν πλέον καλή υποστήριξη JSON τόσο σε επίπεδο datatype όσο και σε επίπεδο SQL.
- ❖ Μετατροπή στο SELECT (MySQL 5.7 και νεότερη)...

- ❑ σε JSON object (ένα object για κάθε εγγραφή-αποτέλεσμα)

```
SELECT JSON_OBJECT( 'name1', fuelCompName,  
                    'xx', JSON_OBJECT('name2', fuelCompNormalName)) as foo  
  
FROM fuelcomps  
WHERE fuelCompID=1
```

- Αποτέλεσμα: {"xx": {"name2": "Α.Π."}, "name1": "ΑΝΕΞΑΡΤΗΤΟ ΠΡΑΤΗΡΙΟ"}
✓ Προσέξτε την ανάποδη σειρά των ζευγαριών στο αποτέλεσμα!

- ❑ σε JSON array (ένα array για κάθε εγγραφή-αποτέλεσμα)

```
SELECT JSON_ARRAY( 'name1', fuelCompName,  
                  'xx', JSON_OBJECT('name2', fuelCompNormalName)) as foo  
  
FROM fuelcomps  
WHERE fuelCompID=1
```

- Αποτέλεσμα: ["name1", "ΑΝΕΞΑΡΤΗΤΟ ΠΡΑΤΗΡΙΟ", "xx", {"name2": "Α.Π."}]

- ❖ Περισσότερα στο JSON Function Reference στο MySQL Documentation:

- ❑ <https://dev.mysql.com/doc/refman/8.0/en/json-function-reference.html>

Κλήση REST API μέσα από PHP

- ❖ Δημιουργία του HTTP αιτήματος (URL + παράμετροι), για παράδειγμα:
 - ❑ `$url=https://api.foursquare.com/v2/venues/venue_id?oauth_token=XX&v=YY`
- ❖ Αποστολή του HTTP και Ανάκτηση Περιεχομένου
`$content = @file_get_contents($url);` //με το @ αποκρύπτεις τυχόν warnings.
- ❖ Μετατροπή σε php array
`$JSONArray = (array)json_decode($content, true);`
- ❖ Εναλλακτικά, χρήση της CURL library της PHP

```
function fetchUrl($uri) {  
    $handle = curl_init();  
    curl_setopt($handle, CURLOPT_URL, $uri);  
    curl_setopt($handle, CURLOPT_SSL_VERIFYPEER, false);  
    curl_setopt($handle, CURLOPT_RETURNTRANSFER, true);  
    $response = curl_exec($handle);  
    $jsonArray= (array)json_decode($response,true);  
    if($jsonArray["meta"]["code"] != 200){  
        echo "error";  
    }else{  
        return $jsonArray;  
    }  
}  
  
$response = fetchUrl($url);
```

HATEOAS

Hypermedia As The Engine Of Application State

- ❖ Ένας client αλληλεπιδρά με μια διαδικτυακή εφαρμογή διαμέσου υπερμέσων (hypermedia) που παράγονται εξ ολοκλήρου στη διαδικτυακή εφαρμογή.
- ❖ **Πλεονέκτημα:** Ο REST client δεν χρειάζεται να ξέρει πώς πρέπει να αλληλοεπιδράσει με κάποιο server ή εφαρμογή πέρα από το να αντιλαμβάνεται και να μπορεί να χειριστεί το περιεχόμενο που λαμβάνει.
 - ❑ Σε αντίθεση, σε άλλες service oriented αρχιτεκτονικές (SOA) όπως π.χ. στην SOAP, clients και servers αλληλεπιδρούν με ρητά δηλωμένες προδιαγραφές (π.χ. Interface Description Language)
- ❖ Θεωρείται ως το τρίτο και τελευταίο επίπεδο οργάνωσης στο REST μοντέλο.
 - ❑ **Επίπεδο 1:** Χειρισμός πολυπλοκότητας με προσέγγιση διαίρει και βασίλευε (σπάσιμο μιας πολύπλοκης υπηρεσίας σε επιμέρους απλούστερες).
 - ❑ **Επίπεδο 2:** Χειρισμός παρόμοιων περιπτώσεων με παρόμοιο τρόπο μέσω της υιοθέτησης ενός τυποποιημένου συνόλου ενεργειών (**GET, PUT, POST, DELETE**).
 - ❑ **Επίπεδο 3:** Εισάγει discoverability (ανακαλυψιμότητα ??), δηλαδή δυνατότητα να κατανοήσεις τον τρόπο λειτουργίας χωρίς να ανατρέχεις στην τεκμηρίωση.

HATEOAS – Παράδειγμα

- ❖ Έστω response σε κάποιο χρονικό σημείο A. Περιέχει και links για τη επόμενη πιθανή ενέργεια του χρήστη, που εννοείται ότι είναι έγκυρη σε αυτό το χρονικό σημείο.

```
<?xml version="1.0"?>
```

```
<account>
```

```
  <account_number>12345</account_number>
```

```
  <balance currency="usd">100.00</balance>
```

```
  <link rel="deposit" href="https://somebank.org/account/12345/deposit" />
```

```
  <link rel="withdraw" href="https://somebank.org/account/12345/withdraw" />
```

```
  <link rel="transfer" href="https://somebank.org/account/12345/transfer" />
```

```
  <link rel="close" href="https://somebank.org/account/12345/close" />
```

```
</account>
```

- ❖ Σε μεταγενέστερο χρονικό σημείο B, το response μπορεί να διαφοροποιηθεί!

```
<?xml version="1.0"?>
```

```
<account>
```

```
  <account_number>12345</account_number>
```

```
  <balance currency="usd">-25.00</balance>
```

```
  <link rel="deposit" href="https://somebank.org/account/12345/deposit" />
```

```
</account>
```




Ασφάλεια: JWT – Jason Web Token

- ❖ Open standard για διακίνηση ασφαλούς πληροφορίας (passing claims) μεταξύ δύο πλευρών.
 - ❑ Self-contained (αυτάρκης): μεταφέρει όλη την απαιτούμενη πληροφορία
 - ❑ Το "πακέτο" πληροφορίας που διακινείται είναι ένα JSON object
- ❖ Χρησιμοποιείται κύρια σε web εφαρμογές (services).
- ❖ Δεν εξαρτάται από κάποια γλώσσα προγραμματισμού. Μπορεί να χρησιμοποιηθεί με PHP, JavaScript (node.js), .NET, python, κτλ.
- ❖ Η διακίνηση μπορεί να γίνει:
 - ❑ ως παράμετρος στο URL (query string)
 - ❑ ως μέρος των POST δεδομένων κάποιας υποβληθείσας (submitted) φόρμας
 - ❑ ως cookie
 - ❑ ως HTTP Header (x-access-token)
- ❖ Καλή λύση για πρόσβαση σε πολλαπλές υπηρεσίες με μοναδικό login.
 - ❑ single sign-on context

Δομή JSON Web Token 1/2

❖ Τρία τμήματα χωρισμένα με τελεία: **header.payload.signature**

❖ **Header**: Συνήθως έχει 2 τμήματα σε κωδικοποίηση JSON

- ❑ **typ**: με τιμή **JWT**
- ❑ **alg**: ο αλγόριθμος hashing που παράγει το **signature** (HS256, ES384, κτλ)

```
{  
  "typ": "JWT",  
  "alg": "HS256"  
}
```

❖ **Payload**: περιέχει τα δεδομένα που θέλουμε να διακινήσουμε, δηλαδή την κυρίως πληροφορία του token.

- ❑ τα δεδομένα (οι ισχυρισμοί/claims των συμβαλλόμενων πλευρών) κωδικοποιούνται κατά JSON σε key:value ζευγάρια
- ❑ Ενδεικτικά keys:
 - **iss**: issuer (εκδότης)
 - **sub**: subject (θέμα)
 - **aud**: audience (σε ποιον απευθύνεται)
 - **exp**: expiration time (πότε λήγει)
 - **nbf**: not before (μετά από ποια ημ/νία ισχύει)
 - **iat**: issued at (πότε εκδόθηκε)
- ❑ **name**, **admin**, κ.α.: custom keys

```
{  
  "iss": "myService Inc.",  
  "name": "fotis",  
  "admin": false  
}
```

Δομή JSON Web Token 2/2

- ❖ Τα header και payload εκτός από JSON δομή, πρέπει να έχουν και BASE64 κωδικοποίηση.
 - ❑ Αυτό σημαίνει ότι αν κάποιος υποκλέψει το token, κάνοντας decode μπορεί να δει το περιεχόμενο των δύο αυτών τμημάτων!
 - ❑ ΑΡΑ το JWT από μόνο του δεν παρέχει ασφάλεια στο περιεχόμενο. Αυτό που παρέχει είναι εγγύηση για την γνησιότητα του περιεχομένου και το κάνει μέσω του τρίτου τμήματος, του **signature**.
 - Αν επιπλέον επιθυμούμε και ασφάλεια (να μη μπορεί να δει κάποιος το payload) θα πρέπει να χρησιμοποιήσουμε κάποιας μορφής κρυπτογράφηση στην μία πλευρά που να επιτρέπει αποκρυπτογράφηση στην απέναντι πλευρά – π.χ. public key encryption)
- ❖ **Signature**: hash των **header** και **payload** μέσω κάποιου password
 - ❑ Σε ψευδοκώδικα:

```
part1 = base64encoded(header) + "." + base64encoded(payload) + "."  
var signature = hashAlgorithm(part1, 'someSecretPassword')
```
- ❖ Άρα: **JWT** = **header** + **payload** + **signature**
 - ❑ Κάθε συναλλασσόμενη πλευρά μπορεί να ελέγξει ότι το περιεχόμενο του token δεν έχει αλλοιωθεί από τρίτο άτομο, απλά επαληθεύοντας το signature με τον γνωστό αλγόριθμο hashing και το γνωστό secret/password του client.

Πώς λειτουργεί στην πράξη;



1. Ζητείται token παρέχοντας στοιχεία login.	2. Έλεγχος εγκυρότητας και δημιουργία JWT.
4. Επαλήθευση token, εξαγωγή πληροφορίας για χρήση, αποθήκευση token (client side) για μελλοντική χρήση.	3. Αποστολή του JWT στον client ή κάποιου error message αν δεν πιστοποιηθεί.
5. Χρήση του αποθηκευμένου token σε μετέπειτα κλήσεις στον server.	6. Έλεγχος εγκυρότητας token σε κάθε μετέπειτα κλήση από τον client.
	7. Αν ο έλεγχος είναι επιτυχής η κλήση διεκπεραιώνετε, αλλιώς επιστρέφεται κάποιο error message.



Σύνδεσμοι για on-line Πειραματισμό

❖ Δημιουργία JWT:

- ❑ <http://jwtbuilder.jamiekurtz.com/> (βλ. επόμενο slide)

❖ Επαλήθευση JWT:

- ❑ <https://jwt.io/>

❖ BASE64 encode/decode:

- ❑ <https://www.base64encode.org/>
- ❑ <https://www.base64decode.org/>

Δημιουργία JWT 1/2

Standard JWT Claims

Issuer

Online JWT Builder

Identifier (or, name) of the server or system issuing the token. Typically a DNS name, but doesn't have to be.

Issued At

2018-01-20T05:53:19.795Z

Date/time when the token was issued. (defaults to now) now

Expiration

2019-01-20T05:53:19.795Z

Date/time at which point the token is no longer valid. (defaults to one year from now) now in 20 minutes in 1 year

Audience

www.example.com

Intended recipient of this token; can be any string, as long as the other end uses the same string when validating the token. Typically a DNS name.

Subject

jrocket@example.com

Identifier (or, name) of the user this token represents.

Additional Claims

Claim Type

Value

GivenName

Johnny



Surname

Rocket



Email

jrocket@example.com



Use this section to define 0 or more custom claims for your token. The claim type can be anything, and so can the value.

If recipient of the token is a .NET Framework application, you might want to follow the Microsoft [ClaimType names](#). You can also use the .NET-oriented claim buttons below.

clear all

add one

add email claim

add name claim (.NET)

add role claim (.NET)

add email claim (.NET)


```
{
  "iss": "Online JWT Builder",
  "iat": 1516427599,
  "exp": 1547963599,
  "aud": "www.example.com",
  "sub": "jrocket@example.com",
  "GivenName": "Johnny",
  "Surname": "Rocket",
  "Email": "jrocket@example.com"
}
```

```
{
  "iss": "Online JWT Builder",
  "iat": 1516427599,
  "exp": 1547963599,
  "aud": "www.example.com",
  "sub": "jrocket@example.com",
  "GivenName": "Johnny",
  "Surname": "Rocket",
  "Email": "jrocket@example.com"
}
```

Key

qwertyuiopasdfghjklzxcvbnm123456

32

HS256 ▼

Create Signed JWT

Copy JWT
to Clipboard

[eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJPbmtpbmUgSldUIEJlYWxkZXIiLCJpYXQiOiJE1MTY0Mjc1OTksImVudCkiOiJkaWQvZm9udC8yMDIwLTA1LTM1IiwiaWF0IjoiMTY0Mjc1OTksImNpdCI6ImFkbGlzc2lnbiJ9.YTtDnBf7qP8A_0H1dOo-2K0r3Uu01tEgF8pYUj10](#)

Επαλήθευση JWT (με HS256)

Encoded

PASTE A TOKEN HERE

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJp
c3MiOiJPbmxbmUgSldUIEJ1aWxkZXIiLCJpYXQiO
jE1MTY0Mjc1OTksImV4cCI6MTU0Nzk2MzU5OSwiYX
VkIjoia3d3LmV4YW1wbGUuY29tIiwic3ViIjoianJ
vY2tldEBleGFtcGxlLmNvbSI6IkdpcdmVuTmFtZSI6
IkpvaG5ueSIsIlN1cm5hbWUiOiJSb2NrZXQiLCJFb
WFpbCI6Impyb2NrZXRAZXhhbXBsZS5jb20ifQ.wXc
eKTP60xA_B-GDiZykTKLGYYNQdW6JYCwFf0oSac
```

Decoded

EDIT THE PAYLOAD AND SECRET (ONLY HS256 SUPPORTED)

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "typ": "JWT",
  "alg": "HS256"
}
```

PAYLOAD: DATA

```
{
  "iss": "Online JWT Builder",
  "iat": 1516427599,
  "exp": 1547963599,
  "aud": "www.example.com",
  "sub": "jrocket@example.com",
  "GivenName": "Johnny",
  "Surname": "Rocket",
  "Email": "jrocket@example.com"
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  TvGI0Ltn6wyVEZegjNTMKVFKTLhBJg7e
) ☐ secret ☐ base64 ☐ encoded
```

✔ Signature Verified

❖ Αν κάποιος αλλοιώσει το token, η υπογραφή δεν επαληθεύεται!